

# Loan Workflow System – End-to-End Design & Approach

## Setup & Execution Guide

To run this DAML-based loan system locally, follow these steps:

### Prerequisites

---

- Install the latest version of the [DAML SDK](#) for your operating system (Windows, macOS, or Linux).
- Recommended: Install VS Code with the DAML extension for better contract development and script execution.

### Getting Started

---

```
# Step 1: Install DAML (if not already installed)
daml install latest
```

```
# Step 2: Set up your project
mkdir loan-system && cd loan-system
daml new loan-workflow --template=create-daml-app
cd loan-workflow
```

```
# Step 3: Replace contents with this project's DAML files
# (LoanApproval.daml, TokenDisbursement.daml, LoanRepayment.daml, etc.)
```

```
# Step 4: Build and compile DAML modules
daml build
```

```
# Step 5: Start sandbox and run scripts
daml start # Or use: daml script --script-name Main:test_YourScript
```

### Files Structure

---

- LoanApproval.daml – Defines loan request and approval flow.
- TokenDisbursement.daml – Manages token minting and disbursement.
- LoanRepayment.daml – Handles partial repayments and contract archiving.
- .daml scripts – Validate each flow through test cases

## Assumptions & Design Decisions

- The bank has full control over approval, disbursal, and repayment rules
- Loan approvals are based on borrower salary capped at 25%
- TokenWallet uses an account-based model for balance updates.
- Repayment logic enforces a 5% minimum threshold to prevent microtransactions.
- Upon full repayment, loans are archived and LoanLimit is replenished.

---

## Document Structure

This document presents a comprehensive design for a three-part DAML-based loan system:

- **Loan Approval Workflow** – Models loan requests and approval/rejection flow
- **Token Disbursement Workflow** – Introduces tokenized fund disbursement with bank-defined limits.
- **Repayment Workflow** – Supports partial repayments, enforces minimum repayments, and enables loan closure.

Each section contains design insights, workflow logic, and example use cases to demonstrate the system's integrity, scalability, and enterprise readiness.

---

# 1. Loan Approval Workflow – Design & Approach

## Overview

This is the first of a three-part Loan Workflow system. The Loan Approval System uses DAML to automate the process of loan requests and decisions, ensuring transparency, role-based access control, and traceability across borrower-bank interactions.

## Objective

- Digitize manual loan approvals.
- Ensure secure, rule-based decision making.
- Improve operational efficiency.

## Design Highlights

- **LoanRequest Template:** Initiated by the borrower. Holds loan details and waits for bank action.
- **Loan Template:** Created by the bank upon approval or rejection, recording the final status.

### Access Control:

- Borrower: Can create requests
- Bank: Can approve/reject
- Both: Sign the final Loan contract

### Workflow

1. Borrower submits a LoanRequest
2. Bank either:
  - Approves → creates a Loan with status "Approved".
  - Rejects → creates a Loan with status "Rejected".
3. All actions are immutable and auditable.

### Example Test Case

- Yami requests \$1000 → Approved
- Akhil requests \$1,000,000 → Rejected
- Bheema requests \$100 → Pending

### DAML Choices Summary

| Action             | Role         |
|--------------------|--------------|
| Create LoanRequest | Borrower     |
| Approve/Reject     | Bank         |
| Sign Loan          | Both parties |

### Summary

This loan approval logic sets the foundation for a modular and compliant lending platform. It supports secure, transparent decision making and integrates easily into a broader financial ecosystem.

---

## 2. Token Disbursement Workflow – Design & Approach

### Overview

The second part of the system introduces token disbursement upon loan approval, governed by loan limits set by the bank. Tokens are minted and credited to a borrower's wallet, ensuring disbursement is within the approved loan limit.

## Objective

- Add loan disbursement via tokens after loan approval.
- Enforce global and individual loan limits for safer lending.
- Track disbursements accurately through smart contracts

## Design Highlights

- **TokenWallet Template:** Represents borrower's wallet. Allows minting (Credit) and spending (Debit) of tokens.
- **LoanLimit Template:** Stores total and individual loan limits to validate approval.
- **LoanRequest Template:** Validates requested amount against salary and remaining limit.
- **Loan Template:** Tracks approved/disbursed amounts and allows controlled disbursement.

## Workflow

1. Bank initializes a LoanLimit contract with a total lending cap.
2. Borrower submits a LoanRequest with loan amount and salary.
3. Bank verifies limit, approves the request, and creates a Loan.
4. Bank disburses funds via Disburse choice, minting tokens to TokenWallet
5. Disbursement is validated against approved loan amount.

## Example Test Cases

- Yami: Salary 10,000 → Requests 10,000 → Rejected
- Akhil: Salary 50,000 → Requests 5,000 → Approved → Disbursed to wallet
- Bheema: Salary 150,000 → Requests 2,000 → Approved
- Shashi: Salary 450,000 → Requests 93,000 → Final request, limit exhausted

## DAML Choices Summary

| Action             | Role                    |
|--------------------|-------------------------|
| Create TokenWallet | Issuer, Owner           |
| Credit/Debit Token | Issuer / Owner          |
| Approve Loan       | Bank (with limit check) |
| Disburse Funds     | Bank / Borrower         |

## Summary

This workflow ensures lending stays within bank-defined boundaries. It connects financial approvals with actual disbursements through tokenization. Built-in assertions and contract roles maintain trust and prevent over-disbursement.

---

# 3. Repayment Workflow – Design & Approach

## Overview

The final part of the loan workflow adds repayment functionality. Borrowers repay disbursed loan amounts incrementally, while adhering to a minimum repayment threshold. The system tracks total repayments and allows automatic loan closure once the full disbursed amount is repaid.

## Objective

- Support partial repayments of disbursed loans.
- Enforce a minimum repayment requirement.
- Automatically close loans upon full repayment

## Design Highlights

- **LoanRepaymentRestriction Template:** Holds the minimum repayment value (5% of disbursed amount).
- **Loan Template:**
  - Adds a Repay choice.
  - Updates repaidAmount on each repayment.
  - Archives the loan if fully repaid.
  - Simultaneously updates the LoanLimit contract's total available balance.
- **Repay Choice:** Ensures amount  $\geq$  minimumPay and  $\leq$  remaining balance.

## Workflow

1. Create a LoanRepaymentRestriction based on 5% of disbursed amount.
2. Borrower exercises Repay, triggering:
  - Validation against minimum
  - Debit from TokenWallet
  - Addition to repaid amount
  - Update of LoanLimit's total limit
3. If full amount is repaid, the loan contract is archived.

## Example Test Case

- Akhil: Loan of 5000  $\rightarrow$  Repays in 3 steps: 500 + 800 + 3700.
- Loan contract is archived after full repayment.
- LoanLimit is updated after each repayment.

## DAML Choices Summary

| <b>Action</b>           | <b>Role</b> |
|-------------------------|-------------|
| Create Restriction      | Bank        |
| Fetch Minimum Repayment | Bank        |
| Repay Loan              | Borrower    |
| Update LoanLimit        | Bank        |

## **Summary**

This workflow completes the lifecycle of a loan by introducing controlled and rule-bound repayments. It enforces minimum repayment, ensures wallet balance usage, and archives contracts upon completion—all while maintaining auditability and balance recovery through LoanLimit updates.